Assignment 02 – ALU

EIT HDL – Doku

Lukas Wolf

2. November 2026

1 Aufgabe

Das Modul `alu` verknüpft zwei 16-Bit-Werte `x` und `y`. Welche Funktion berechnet wird, bestimmen die sechs Steuerbits `zx, nx, zy, ny, f, no`. Zusätzlich liefert die ALU die Flags `zr` (`out == 0`) und `ng` (`out < 0`).

2 Implementierung

Die ALU ist in einem `always_comb`-Block umgesetzt, Schritt für Schritt wie in der Angabe: zuerst x und y vorbereiten, dann addieren oder UND-verknüpfen, am Ende eventuell invertieren.

```
always_comb begin
    x_zx = zx ? '0 : x;
    x_nx = nx ? ~x_zx : x_zx;
    y_zy = zy ? '0 : y;
    y_ny = ny ? ~y_zy : y_zy;
    f_out = f ? (x_nx + y_ny) : (x_nx & y_ny);
    no_out = no ? ~f_out : f_out;
end

assign out = no_out;
assign zr = (out == '0);
assign ng = out[W-1];
```

3 Verifikation

Die Testbench legt die 9 geforderten Operationen im Abstand von 100 ns an und vergleicht `out`, `zr` und `ng` mit dem erwarteten Wert. Fehler werden gezählt, am Ende wird PASS oder FAIL ausgegeben. Gestartet wird die Simulation in ModelSim mit `do sim_tb_alu.tcl`.

Steuerbits für `0xaa | 0x55` (`010101`): `nx=1` und `ny=1` invertieren x und y, `f=0` verknüpft sie mit UND, `no=1` invertiert das Ergebnis. Nach De Morgan gilt $\overline{\overline{x}\,\&\,\overline{y}} = x \mid y$.

Simulator-Ausgabe

```
OK   1 + 2          cmd=000010 | out=     3 (0x0003) zr=0 ng=0
OK   100 - 50       cmd=010011 | out=    50 (0x0032) zr=0 ng=0
OK   50 - 100       cmd=010011 | out=   -50 (0xffce) zr=0 ng=1
OK   0              cmd=101010 | out=     0 (0x0000) zr=1 ng=0
OK   0xaa & 0x55    cmd=000000 | out=     0 (0x0000) zr=1 ng=0
OK   0xaa | 0x55    cmd=010101 | out=   255 (0x00ff) zr=0 ng=0
OK   -1             cmd=111010 | out=    -1 (0xffff) zr=0 ng=1
OK   !255           cmd=001101 | out=  -256 (0xff00) zr=0 ng=1
OK   -255           cmd=001111 | out=  -255 (0xff01) zr=0 ng=1
-----------------------------------------------
PASS: alle Tests bestanden
-----------------------------------------------
```

Wave-Fenster

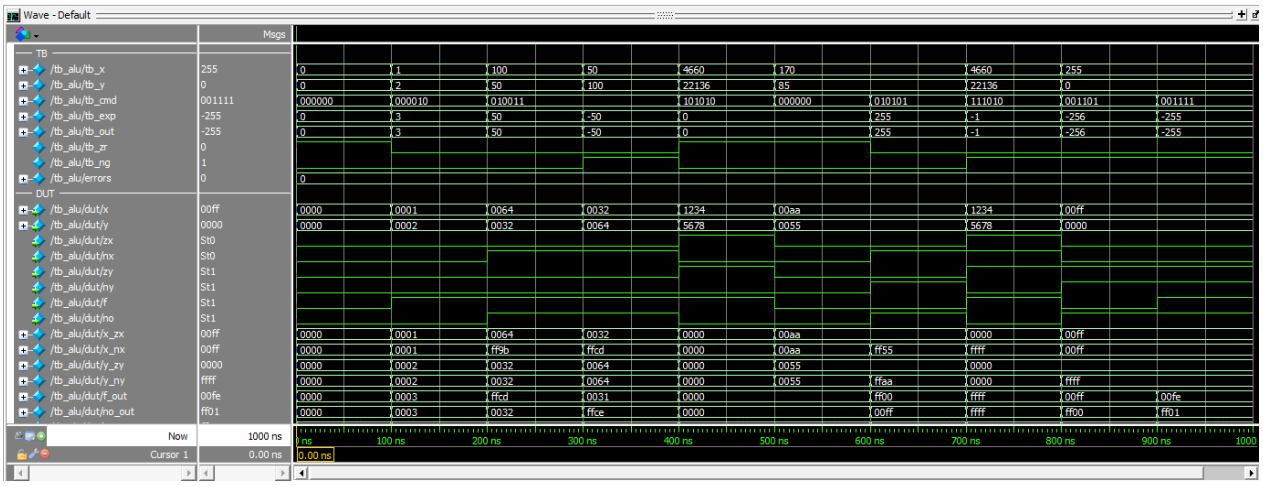


Abbildung 1: ModelSim Wave-Fenster der Simulation von `tb_alu`.

Ergebnis Alle 9 Operationen liefern das erwartete Ergebnis, der Fehlerzähler bleibt 0. Das Modul erfüllt damit die Spezifikation.

4 Diskussion der Implementierungsart

Methode	Vorteile	Nachteile
Gatter	Man bestimmt jedes einzelne Gatter selbst.	Für 16 Bit und einen Addierer sehr viel Code, fehleranfällig und schwer zu lesen.
`case` über alle Funktionen	Jede Funktion steht direkt im Code.	Viele Zeilen, entspricht nicht dem Aufbau aus der Angabe.
`always_comb` mit Stufen (gewählt)	Kurzer Code, genau wie in der Angabe aufgebaut.	Welche Funktion berechnet wird, sieht man nur mit der Tabelle aus der Angabe.